

Deadlock ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է deadlock-ի՝ ինչ է, 4 paymany, real orinak (sxal + chisht lucum), inchpes xusapel, ev interview harcer.

1. Ի՞նչ է Deadlock-ը

Deadlock-ի լինում է, երբ **erku (kam aveli) thread spasum en irar** anverjoren, u voch meky chi karoxtanum aʻraj gnal.

```
Thread A: pahum e Lock1, spasum e Lock2
Thread B: pahum e Lock2, spasum e Lock1
-> erku-n el spasum en -> KAXVAC (deadlock)
```

2. Real orinak (SXAL kod)

```
std::mutex m1, m2;

void threadA() {
    std::lock_guard<std::mutex> l1(m1);    // pahum m1
    std::lock_guard<std::mutex> l2(m2);    // spasum m2
}

void threadB() {
    std::lock_guard<std::mutex> l1(m2);    // pahum m2
    std::lock_guard<std::mutex> l2(m1);    // spasum m1
}

// A pahum m1 spasum m2; B pahum m2 spasum m1 -> DEADLOCK
```

Problem-ի **tarber kargov** en lock anum (A: m1->m2, B: m2->m1)。

3. Deadlock-i 4 paymany (Coffman conditions)

Deadlock linelu hamar petq en **bolor 4-y** miasin.

1. Mutual Exclusion -> resource-y miayn mek thread miangamic
2. Hold and Wait -> thread pahum e mek, spasum myusin
3. No Preemption -> resource-y brnov chi arvum (kamovin veradarcnum)
4. Circular Wait -> A spasum B-in, B spasum A-in (shrjan)

Meky kotres -> deadlock chi lini.

4. Lucum 1 — Nuyn kargov lock

```
void threadA() {
    std::lock_guard<std::mutex> l1(m1);    // m1 a'raj
    std::lock_guard<std::mutex> l2(m2);    // heta m2
}
void threadB() {
    std::lock_guard<std::mutex> l1(m1);    // NUYN karg: m1 a'raj
    std::lock_guard<std::mutex> l2(m2);    // heta m2
}
// erku-n el nuyn kargov -> circular wait kotrvum -> CHKA deadlock
```

Amenapařz lucumy. **misht nuyn kargov lock ani mutex-nery.**

5. Lucum 2 — std::lock (atomic mi qani lock)

```
void threadA() {
    std::lock(m1, m2);    // erku-n el miasin lock (deadlock-free)
    std::lock_guard<std::mutex> l1(m1, std::adopt_lock);
    std::lock_guard<std::mutex> l2(m2, std::adopt_lock);
}
```

`std::lock` -y erku mutex-y lock e anum atomic (deadlock-ic azat algoritmov). C++17-um `std::scoped_lock`.

```
std::scoped_lock lock(m1, m2); // C++17 -- mi qani mutex, deadlock-free
```

6. Lucum 3 — try_lock (timeout)

```
std::timed_mutex m;  
if (m.try_lock_for(std::chrono::seconds(1))) {  
    // ashxatanq  
    m.unlock();  
} else {  
    // chstacav -> ayl banov shrjanci (deadlock-ic xusapel)  
}
```

7. Xusapelu dzevery — амфотф

1. Misht NUYN KARGOV lock ani mutex-nery (amenaparʼz)
2. std::lock / std::scoped_lock (atomic mi qani lock)
3. try_lock timeout-ov
4. Qich mutex, poqr critical section
5. Chpahel lock erkar (I/O lock-i mej che)

8. Interview harcer & patasxanner

Deadlock inch e?

Erku+ thread spasum en irar anverjoren; voch meky araj chi gnum (kaxvac).

Inchu e linum?

Thread-ner tarber kargov mi qani lock -> circular wait.

4 paymannery?

Mutual exclusion, hold and wait, no preemption, circular wait (Coffman). Bolor 4-y petq en.

Inchpes lucel/xusapel?

Nuyn kargov lock; std::lock/scoped_lock (atomic); try_lock timeout.

std::lock inch e anum?

Mi qani mutex lock e anum atomic, deadlock-free algoritmov.

scoped_lock (C++17)?

Mi qani mutex, RAII, deadlock-free (std::lock + lock_guard miasin).

Circular wait inchpes kotrel?

Global karg sahmanel mutex-neri hamar, misht ayd kargov lock.

Cheat Sheet

DEADLOCK = erku+ thread spasum en irar anverjoren (kaxvac)

INCHU: tarber kargov mi qani lock -> circular wait

4 PAYMAN (Coffman, bolor 4 petq en):

Mutual Exclusion | Hold and Wait | No Preemption | Circular Wait

LUCUM:

1. NUYN KARGOV lock (amenapařz)
2. std::lock / std::scoped_lock (C++17, atomic mi qani)
3. try_lock timeout-ov
4. qich mutex, poqr critical section

KANON: global lock ordering -> circular wait kotrvum